

WHAM-XREX-PFMG₄₇₄

Firmware Build, Flash, and Verification

Standard Operating Procedure

August 2026

Rev	Date	Author	Description
A	2026-08-31	–	Initial issue: build, bootstrap flash, serial reflash. No power-stage procedures yet (Section 2).
B	2026-09-04	–	Added Section 8: uploading a PFM table and firing it – this firmware now drives real HRTIM PWM output (Section 2 updated accordingly). Serial baud raised 9600 → 115200 throughout (Section 7 and elsewhere).
C	2026-09-22	–	Added Section 9: the modern closed-loop shot-profile procedure (ARM/SHOT:*), driven by hand over raw SCPI, bypassing <code>wham_console.py</code> 's shot wizard. Corrected Section 2's Rev B claim that this firmware has “no ARM precondition, no state machine, no fault interlock” – all three now exist (added between Rev B and Rev C; Section 8's own caution still applies as-is to the legacy TABLE:*/FIRE path specifically, which remains interlock-free by design).

Rev	Date	Author	Description
D	2026-09-22	–	Added Section 10: reading and translating the !EVT telemetry stream and the retained flight-recorder log (SYS:EVLOG?). Updated Section 9: the external-enable (EXTernal:ENABle, PF13) and external-trigger (EXTernal:TRIGger, PF15) interlocks now default ON at every boot (were opt-in/OFF through Rev C), a real behavior change for this procedure’s own ARM step on hardware with nothing wired to PF13; and ARM’s failure reporting was reworked to name the specific failed precondition (which channel, which interlock) instead of one generic error. Documented firing via the external trigger as an alternative to Step 8’s manual SHOT:START. Updated Section 12 to match.
E	2026-09-22	–	Wire-protocol rename: the output/demand commands moved out of PID:* into SOURce:*/SHOT:*/LOG:*/CHANnel:* (clean cut, no aliases); PID:* now means only PID:GAINS/PID:LOOPMODE. All command names in this document updated accordingly.

Rev	Date	Author	Description
F	2026-09-23	–	Added Section 11: a complete reference inventory of every physical signal the controller drives (OUTPUT) and reads (INPUT) at its Transrex- and operator/facility- facing boundary – per-channel signals (DRIVE/FEEDBACK/ENA_OUT/CONTACT_OUT/the three gate-driver fault inputs/OCP), the system-wide interlock/trigger/hardware-fault inputs (PF13/PF15/PC10), and the bench-only diagnostic loopback outputs. Includes a direct caution that ENA_OUT/ CONTACT_OUT are commands only, with no physical confirmation signal back from the Transrex – SM_FAULT_ENABLE_OUTPUT detects the controller’s own commanded GPIO state changing, not whether the Transrex’s contactor actually responded.

1 Purpose

This document specifies the procedure for building, flashing, and verifying WHAM-XREX-PFMG474 firmware, and for confirming a board is running the expected firmware version before it is put into service or handed off.

2 Scope

Covers firmware build and flashing (Sections 4–7), the legacy PWM-table shot path (Section 8, Rev B), and, as of Rev C, the modern closed-loop shot-profile path (Section 9) – this firmware **drives real HRTIM PWM output** the moment FIRE (legacy path) or SHOT:START (modern path) is sent.

Correction, Rev C: Rev B stated this firmware had “no ARM precondition, no state machine, and no fault interlock.” The ARM/ state-machine part was accurate for Rev B’s own scope (the legacy TABLE:*/FIRE path *still* has no ARM step and no state machine, by design – Section 8’s caution covers this) – but the “no fault interlock” part was already incomplete even at Rev B (FIRE has always rejected a latched fault with ERR 6, see Section 8’s own caution, now corrected). A full state machine (IDLE/ARMED/FIRING/FAULT), an ARM precondition, and a richer, per-category fault interlock set (overcurrent, water/temp, Enerpro, enable-

output) were added to the *modern* path between Rev B and Rev C – see Section 9. Treat any gate driver or power stage wired to these pins as live before either procedure, per your lab’s normal HV/gate- driver procedure – the modern path’s interlocks protect against several real fault conditions but this SOP still provides no substitute for that normal procedure. This document still does not cover power- stage bring-up or calibration.

3 Prerequisites

3.1 Equipment

Item	Notes
WHAM-XREX-PFMG474 board	Powered per its own hardware documentation (not covered here).
ST-Link (or equivalent SWD debug probe)	Required for the bootstrap flash (Section 5) and for recovery if a board stops responding over serial entirely.
USB-to-serial adapter	Wired to USART2: adapter TX → PA3 (USART2_RX), adapter RX → PA2 (USART2_TX), GND ↔ GND.

(TX/RX wiring above is crossed: adapter TX goes to the MCU’s RX pin and vice versa, as usual for a point-to-point serial link.)

3.2 Software

Tool	Notes
STM32CubeIDE stm32flash	Project build and the bootstrap SWD flash. AN3155 USART bootloader client. macOS: brew install stm32flash.
Python 3 + pyserial	Runs python/wham_serial_flash.py. pip install pyserial.

CAUTION: This firmware has not yet been through an ESD-controlled production process. Use standard ESD precautions (grounding strap, static-safe surface) when handling the bare board.

4 Procedure: Build the Firmware

1. Open the project in STM32CubeIDE and press **Refresh** (F5) on the project if any files were added outside the IDE since it was last opened.

2. Build the **Debug** configuration (Project → Build Project, or from the command line: `cd Debug && make all -j4`).
3. Confirm the build produced no warnings. A clean build produces `WHAM-XREX-PFMG474.elf` in `Debug/`.
4. **Generate the .bin.** This project’s build configuration does not have the “Convert to binary file” post-build step enabled, so it must be generated by hand:

```
arm-none-eabi-objcopy -O binary \
  Debug/WHAM-XREX-PFMG474.elf \
  Debug/WHAM-XREX-PFMG474.bin
```

5 Procedure: Bootstrap Flash (SWD)

Required the *first* time a board receives firmware with the serial-bootloader feature (BOOT command), since that feature is what the serial reflash path in Section 6 depends on – a board cannot use it to receive the very first firmware that adds it. Also use this procedure to recover a board that no longer responds to BOOT over serial at all.

1. Connect the ST-Link to the board’s SWD header and to the host computer.
2. In STM32CubeIDE, press **Run** (or **Debug**) to build and flash via the ST-Link, as normal for this IDE.
3. Proceed to Section 7 to confirm the board is running the expected firmware.

6 Procedure: Serial Reflash

Use this for all firmware updates *after* the bootstrap flash in Section 5 has been done once. This is the remote-friendly path: no physical BOOTo/NRST/SWD access is required, only the USART2 serial link.

1. With the board powered and running its current firmware, and the USB-serial adapter connected, run:

```
python3 python/wham_serial_flash.py \
  --port /dev/cu.usbserial-XXXXX
```

substituting the actual serial device path. (On macOS, a single USB-serial adapter shows up as both a `cu.*` and `tty.*` device node; the script sees this as two candidates and will not auto-detect – `--port` must be passed explicitly. Prefer the `cu.*` node.)

2. The script will: send `BOOT` and wait for the acknowledgement; run `stm32flash` to write and verify `Debug/WHAM-XREX-PFMG474.bin`; then start the new firmware automatically (no manual reset needed – see Section 6.1).
3. Confirm the script reports `[done]` with firmware written and verified before proceeding.
4. Proceed to Section 7.

6.1 Note on automatic restart after flashing

Early testing of this procedure found that the newly-flashed firmware could come up unresponsive over serial until the board was manually power-cycled, because the STM32 ROM bootloader's *Go* command (used to start the new firmware without a physical reset) does not perform a true chip reset, and left the processor's vector table pointer aimed at the bootloader's own vector table rather than the application's. This has been fixed in firmware (`boot_jump.c`) – a manual reset should not be necessary. If a board flashed under this SOP does not respond to the verification step below without a manual reset, that is a regression and should be reported, not treated as expected behavior.

7 Procedure: Verification

1. Open a serial connection to the board at **115200 8N1** (raised from 9600 as of Rev B – see `docs/changelog.txt`).
2. Send `*IDN?` followed by a line ending (`\r`, `\n`, or both).
3. Confirm a reply of the form:

`OK <board name> <hardware rev> <firmware version>`
 e.g. `OK WHAM-XREX-PFMG474 REVA v0.1.`
4. Confirm the reported firmware version matches the version that was just built/flushed.

Acceptance criterion for this SOP: Step 3 above returns a well-formed reply with the expected version, on the first query, with no manual reset performed since the flash in Section 4 or 6.

8 Procedure: Upload and Fire a PFM Table

New in Rev B. Uploads a complete PWM shot profile (a sequence of period/duty entries, one per switching cycle) and plays it back on demand. Full protocol detail: docs/command_reference.md's TABLE:* and FIRE entries.

CAUTION: FIRE drives real HRTIM PWM output on six pins (three complementary phase pairs, ~ 100 ns hardware dead time between each pair) the instant it is sent. There is still no ARM step and no state machine on *this legacy path* as of Rev C – FIRE always takes effect immediately once its two checks pass (below), whether the board was idle or a previous shot was still running; the ARM/state-machine system added since Rev B applies only to the modern SHOT:* path (Section 9), not to FIRE. **Correction, Rev C:** FIRE DOES check for a latched fault (ERR 6 if one is, FAULT:CLEAR first) in addition to the empty-table check below – Rev B's "no fault interlock" claim for this path was incomplete even at the time and is corrected here; this is still a single blanket check (any latched fault, not per-category), not the modern path's richer interlock set. Treat any gate driver or power stage wired to these pins as live before proceeding, per your lab's normal HV/gate-driver procedure.

1. Complete Sections 4–7 first – a board must already be running current, verified firmware.
2. Table *construction* (what shot profile to build) lives entirely on the host, not in firmware – see pfm.h's own header note for why. python/pfm_table_upload.py is the reference builder: edit its PROFILE constant (1–5; see the script's own header comment for what each one is) before running:

```
python3 python/pfm_table_upload.py \  
--port /dev/cu.usbserial-XXXXX
```
3. Confirm the script reports [done] table uploaded and confirmed with the expected entry count (TABLE:END -> OK <count>).
4. Optionally confirm independently at any time with TABLE? over a serial terminal (e.g. python/scpi.py) – replies OK <count>, matching the upload.
5. With the CAUTION above acknowledged and anything downstream of these six pins confirmed safe to drive, send FIRE. A prompt OK confirms the shot started; output stops on its own, at a coherent period boundary, once the table is exhausted – there is no separate STOP command and no completion notification.

6. Sending FIRE against an empty table (nothing uploaded yet) is rejected with ERR 5 and drives nothing; sending it with a fault latched is rejected with ERR 6 (send FAULT: CLEAR first) – these two checks are this path’s only built-in safeguards, not a substitute for the interlock discussed above.

Acceptance criterion for this SOP: TABLE:END reports the exact entry count the host script intended to send, and FIRE returns OK with no ERR. Confirming the actual output waveform (dead time, phase spacing, duty accuracy) is a bench/instrumentation task outside this SOP’s scope – see docs/test_protocol.md’s T4 group if that level of verification is needed.

9 Procedure: Program and Fire a Shot Profile (Raw SCPI)

New in Rev C. The modern shot path – ARM/SHOT:* – driven entirely by hand over a bare serial terminal, for an operator who wants to understand (or reproduce without any host tooling) exactly what python/wham_console.py’s guided shot wizard does under the hood. **For routine bench work, use the shot wizard instead** (python3 python/wham_console.py, then shot – it applies every step below automatically, with prompts, a pre-fire summary, and automatic plotting) – this procedure is for when that tooling is unavailable, being debugged itself, or an operator specifically wants direct command-by-command control. Full protocol detail for every command below: docs/command_reference.md’s SOURCE:*/SHOT:*/LOG:*/CHANNEL:* and XREX:* entries.

CAUTION: SHOT:START (Step 8a) drives real HRTIM PWM output on every enabled channel, ramping toward that channel's own commanded demand current, from the moment it returns OK. Unlike the legacy path (Section 8), this path DOES have a state machine (IDLE → ARMED → FIRING, back to IDLE automatically at shot end or on a fault) and DOES check real preconditions before ARM succeeds (Step 7) – but those preconditions are about channel readiness and interlock state, not about whether a human has actually confirmed the downstream hardware is safe to drive. Treat any gate driver or power stage wired to an enabled channel's output as live before Step 8, per your lab's normal HV/gate-driver procedure, exactly as for the legacy path.

If firing via the external trigger instead (Step 8b, Rev D): the board stays ARMED – and will fire on the very next PF15 rising edge – for as long as it takes an operator to reach for that trigger, which may be well after Step 7. Treat ARMED with EXTERNAL:TRIGGER on as “could fire at any moment,” not just “about to fire right now”: confirm the downstream hardware is safe BEFORE sending ARM in this mode, not just before the trigger pull, and DISARM promptly if the trigger will not be pulled soon after arming.

1. Open a serial connection at **115200 8N1** and confirm the board responds to *IDN? (as in Section 7).
2. Confirm the starting state with STATE? – expect OK IDLE. If it instead reports OK FAULT <type>, resolve the underlying condition first, then send FAULT:CLEAR (a fault cleared without resolving its real cause will simply re-latch, for every fault type this firmware detects).
3. **For each channel to be used this shot**, send, in this order (every value explicit – this firmware never resets a channel's configuration between shots on its own, so anything not sent here keeps whatever a previous shot, or a previous operator, left it at):
 - (a) XREX:CHANnel:ENAOOut <ch> 1
 - (b) XREX:CHANnel:CONTactOut <ch> 1
 - (c) SOURce:ENABle <ch> 1
 - (d) PID:LOOPMODE <ch> <0|1> – 0 = open-loop (profile value written straight to output, no correction), 1 = closed-loop.
 - (e) *Closed-loop only:* PID:GAINS <ch> <Kp> <Ki> <Kd>.
 - (f) SHOT:CURREnt <ch> <demandCurrentA> – this channel's peak current for the shot (clamped to its own configured full-scale current).

4. **For every channel NOT being used this shot**, explicitly disable it: `SOURce:ENABle <ch> 0`, `XREX:CHANnel: ENAOut <ch> 0`, `XREX:CHANnel:CONTactOut <ch> 0`. Step 8 drives *every currently-enabled* channel, not just the ones just configured in Step 3 – a channel left enabled from an earlier shot will fire too, toward whatever demand current it was last given.
5. Set the shared shot timing (one synchronized clock for every channel): `SHOT:TIMing <rampUpTimeS> <flatTopTimeS> <rampDownTimeS>` – three independent durations as of Rev C (all three required, each must be > 0 ; ramp up and ramp down no longer have to match).
6. *Optional but recommended*: arm waveform logging, so the shot's actual behavior can be inspected afterward: `LOG:ARM <ch> <maxSamples> <decim>` for one channel, or `LOG:ARM 0 <maxSamples> <decim>` for every enabled channel at once, from the same real ticks.
7. Send ARM. A plain OK confirms the transition to ARMED. On failure, the reply now names the specific precondition that failed (reworked Rev D – previously one generic ERR 13 covered every cause): ERR 13 Can't ARM -- not currently IDLE (clear a fault or DISARM first); ERR 15 External enable interlock not satisfied -- PF13 reads LOW (see the EXTERNAL:ENABle note below); or ERR 13 Channel N's ENA_OUT/CONTACT_OUT not both set (fix that channel per Step 3a/3b, or disable it per Step 4). Only the *first* failing precondition is reported per attempt – fix it and re-send ARM to see the next one, if any. Send DISARM at any point before Step 8 to cancel back to IDLE without firing anything.

EXTERNAL:ENABle **default, Rev D**: this interlock (PF13) now defaults **ON** at every boot (was opt-in/OFF through Rev C). On hardware with a real external-enable signal wired to PF13, this Just Works. On a bare bench setup with nothing wired there, PF13's resting level depends on the specific hardware – some fiber receivers idle HIGH with no signal present (interlock satisfied by default), a genuinely floating pin reads LOW via the board's own pull-down (interlock blocks ARM until either the real signal is wired, looped back via DIAGNOSTIC:GPOut11 (below), or EXTERNAL:ENABle 0 is sent first for this session). Check EXTERNAL:INPut? to see PF13's actual current level before assuming which case applies.

8. With the CAUTION above acknowledged, fire the shot – either manually (send `SHOT:STARt`) or, **new Rev D**, by physical trigger:
 - (a) *Manual*: send `SHOT:STARt`. OK confirms the shot has begun – every enabled channel begins its own linear ramp (0 A $\rightarrow$ its own

demandCurrentA → flat-top → 0 A) from the same synchronized instant. ERR 12 means SHOT:TIMing was never successfully sent this boot (Step 5); ERR 15 means the external-enable interlock dropped between ARM and this command.

- (b) *External trigger:* send EXTERNAL:TRIGger 1 (defaults ON as of Rev D, same as EXTERNAL:ENABle above – confirm with EXTERNAL:TRIGger? if unsure). While ARMED, a genuine **LOW-to-HIGH transition** on PF15 fires the shot exactly as SHOT:START would – **edge-triggered, not level-triggered**: a signal already HIGH at the moment ARM succeeded does *not* count, even if it stays HIGH afterward; only a fresh transition after arming fires it. Check EXTERNAL:TRIGger:INPut? to see PF15’s current level, and see Section 10 for how to confirm the fire actually happened (a triggered shot emits its own !EVT ... TRIGGER FIRING line, distinct from a command-initiated one) – unlike Step (a), a trigger that fails to fire (wrong state, timing not set, interlock dropped) reports nothing back over serial; it just silently stays ARMED.
9. Wait for completion: poll SOURce:STATus? <ch> (its running field drops to 0), or simply wait rampUpTimeS + flatTopTimeS + rampDownTimeS plus a small margin. The shot ends itself (a full stop, output off entirely – not a hold-at-floor) with no separate STOP command and no completion notification; the state machine returns to IDLE on its own.
10. If logging was armed, fetch it: LOG:DATA? <ch> – replies OK <count> <rateHz> then count groups of setpointHz measuredHz outputHz, one per logged sample.
11. Return the board to a known-safe state before the next shot – Step 9’s automatic stop does *not* do this: for every channel used, SOURce:ENABle <ch> 0, XREX:CHANnel:ENAOut <ch> 0, XREX:CHANnel:CONTactOut <ch> 0, PID:LOOPMODE <ch> 0, PID:GAINS <ch> 0 0 0.

Acceptance criterion for this SOP: SHOT:START returns OK with no ERR; SOURce:STATus? shows running=1 immediately afterward and running=0 once the total shot duration has elapsed; and, if logging was armed, LOG:DATA? reports the same sample count that was requested (or fewer only if the shot itself was shorter than the log’s own capacity). Confirming the actual output waveform against the commanded profile (ramp linearity, flat-top accuracy, closed-loop convergence) is a bench/instrumentation task outside this SOP’s scope – python/wham_console.py’s own plot/report commands, or python/run_simulator_validation.py against the bench simulator, cover that.

10 Procedure: Telemetry – Reading and Translating the Event Stream

New in Rev D. The controller emits **unsolicited** !EVT lines – device-initiated, not sent in response to any command – any time the top-level state (IDLE/ARMED/FIRING/ FAULT) changes, so an operator (or a host script) can observe what the board is doing in real time instead of having to continuously poll STATE?/FAULT?. A separate, always-on retained log (the **flight recorder**) keeps the same history for post-mortem review after the fact, even if nobody was watching the live stream when it happened. Full protocol detail: docs/command_reference.md’s “Unsolicited telemetry events” section and docs/telemetry.md.

10.1 The live stream

Enabled by default (SYS:EVENT? reports 1). A !EVT line can arrive interleaved between any two commands’ own OK/ ERR replies – the !EVT prefix can never collide with either, so it is always unambiguous which kind of line just arrived. python/wham_console.py handles this automatically: every !EVT line is printed inline, tagged with the host’s own wall-clock time, the moment it arrives – nothing special has to be sent or typed to see it happen. Driving the board over a bare terminal instead, the same lines simply appear on their own between typed commands and replies; no special terminal mode is needed. To silence the live stream without losing anything (the flight recorder, below, keeps recording regardless): SYS:EVENT 0. Re-enable with SYS:EVENT 1.

10.2 Translating an !EVT line

Every line has the same shape: !EVT <tick_ms> <rest>. <tick_ms> is a monotonic millisecond counter since this boot (the same clock SYS:TIME? reports on demand) – **not** a wall-clock time; this board has no RTC. To relate an event’s timestamp back to a real time of day, note the host’s own wall-clock time alongside a SYS:TIME? query close in time to the event and work from the difference – wham_console.py’s own inline tagging (above) already does this automatically for the live stream.

Line	Meaning
STATE IDLE	Boot, a shot completed on its own, DISARM, SOURCE:STOP, or a successful FAULT:CLEAR.
STATE ARMED	A successful ARM (Section 9, Step 7).
STATE FIRING	Output has actually begun – either SHOT:START sent manually, <i>or</i> the external trigger (below). This line alone does not distinguish which.
TRIGGER FIRING	Emitted as an EXTRA line, immediately alongside a STATE FIRING caused specifically by a PF15 rising edge (Section 9, Step 8b) – this is how to tell a trigger-initiated shot apart from a command-initiated one: a STATE FIRING with no accompanying TRIGGER FIRING was started by SHOT:START (or SOURCE:RUN); one with it was started by the physical trigger.
FAULT <type> [ch]	A fault was just latched – <type> is one of GENERAL (Water/Temp, or either hardware fault source, system-wide), OVERCURRENT, ENABLE_OUTPUT, or ENERPRO (all three per-channel, [ch] 1-based and present only for these), or EXTERNAL_ENABLE (system-wide, PF13 dropped while FIRING). See Section 9's Scope correction and docs/command_reference.md for what each type actually does to the output (ramp-down vs. immediate stop, system-wide vs. single-channel).
FAULT CLEAR	A successful FAULT:CLEAR – the underlying condition was confirmed actually gone, not merely requested; a FAULT:CLEAR that fails to actually clear (condition still present) produces no line at all, since nothing changed.

10.3 The flight recorder (post-mortem)

SYS:EVLOG? replays every retained event since this boot, oldest first, as a header OK <n> followed by n !EVT lines in the exact same format as the live stream above – the same translation table applies unchanged. Retains up

to 128 entries (overwrite-oldest once full); RAM-only, so it clears on reset like every other runtime state this firmware keeps. Critically, **this keeps recording even with the live stream silenced** (SYS:EVENT 0) or with nobody connected/watching at all – so the first diagnostic step after “something unexpected happened and I don’t know why” should usually be SYS:EVLOG?, not a guess.

Worked example. A real session this same day: an operator found the controller unexpectedly in STATE? OK FAULT GENERAL with no memory of having caused it, and GDS? (the raw gate-driver pin snapshot) showing every pin already healthy – suggesting whatever tripped it had since cleared on its own, but the *latch* (by design, see Section 8’s original fault-interlock discussion) does not clear itself. SYS:EVLOG? resolved it immediately:

```
> SYS:EVLOG?  
< OK 3  
< !EVT 40344 STATE ARMED  
< !EVT 40354 STATE IDLE  
< !EVT 275847 FAULT GENERAL
```

This showed the fault fired spontaneously ~4 minutes *after* the last ARM/IDLE activity, while genuinely IDLE – ruling out a shot-in-progress cause and pointing instead at a real, transient signal edge on the fiber-fed Water/Temp inputs (in this case, traced to another board’s own transmitter glitching during an unrelated reset). FAULT:CLEAR then succeeded cleanly, since GDS? had already confirmed the real condition was gone. Without the flight recorder, the only available facts would have been “it’s faulted now” and “the pins look fine now” – not enough, on their own, to tell a real-but-resolved event apart from a false latch.

11 Reference: Controller I/O – Signals Sent To and Received From the Transrex

New in Rev F. A complete inventory of the physical signals this controller drives (**OUTPUT**) and reads (**INPUT**) at its Transrex-facing and operator/facility-facing boundary – everything in this section is real hardware I/O, not a wire-protocol command. Authoritative source: docs/pin_mapping_v4.csv; every entry below was cross-checked against it and against the firmware source directly, not assumed from memory.

11.1 One thing to be careful about before reading the tables

CAUTION: `XRn_ENA_OUT` and `XRn_CONTACT_OUT` are **OUTPUTS ONLY** – the controller has no way to confirm the Transrex actually responded. These two signals (Section 11.2 below) are commands this firmware sends outward; there is no corresponding status/feedback input anywhere in the pin mapping. The firmware’s own design comment is direct about this: “these are OUTPUTS this firmware commands, not hardware-driven INPUTS it reads – ‘checking’ them is a readback of this firmware’s own last-written GPIO state..., not an interpretation of external signal polarity” (`Core/Inc/xrex_io.h`). Concretely: `SM_FAULT_ENABLE_OUTPUT` (Section 9) detects “did this MCU’s own GPIO register stop reading what we told it to be” – it **CAN-
NOT** detect “the controller is still commanding the contactor, but the Transrex’s contactor itself failed to close or opened unexpectedly.” If physical contactor confirmation matters for your operation, this is a real, currently-uncovered gap, not a case this fault type happens to also catch – treat every “contactor is closed” assumption downstream of `XREX:CHANnel:CONtactOut?/ENAOut?` as “the controller believes it commanded this,” not as physical confirmation.

11.2 Per-channel signals (XR1–XR4)

Eight signal types, one instance per Transrex channel (4 channels on this build – confirm with `CONFig:CHANnels?`). Pin letters below follow `docs/pin_mapping_v4.csv`’s own channel order (`XR1/XR2/XR3/XR4`); wire commands use 1-based channel numbers (`XR1` = channel 1).

Signal (direction)	Pins (XR1–4)	Purpose / commands
<code>XRn_DRIVE</code> (OUTPUT)	<code>PA8</code> , <code>PA10</code> , <code>PB12</code> , <code>PB14</code>	Real PFM switching waveform to the Transrex (<code>HRTIM1_CHA1–CHD1</code>) – the actual power-stage drive signal. Driven the instant <code>FIRE</code> (legacy, Section 8) or <code>SHOT:STARt/SOURce:RUN</code> (modern, Section 9) succeeds.

Signal (direction)	Pins (XR1-4)	Purpose / commands
XRn_FEEDBACK (IN-PUT)	PA15, PD4, PB2, PC12	Measured output frequency (TIM2/TIM5 capture), from the Transrex's own real PFM response – this is what SOURce:STATus?'s measuredHz field and closed-loop PID correction (PID:LOOPMODE) are built on. A genuine hardware input, unlike the two below.
XRn_ENA_OUT (OUT-PUT)	PG0–PG3	Commands the Transrex's own enable line (XREX:CHANnel:ENAOut <ch> <0 1>); both this and CONTACT_OUT below must be 1 for a channel before ARM will succeed for it. See the caution above – output only, no confirmation.
XRn_CONTACT_OUT (OUTPUT)	PG4–PG7	Commands the Transrex's own contactor (XREX:CHANnel:CONtactOut <ch> <0 1>). See the caution above – output only, no confirmation.
XRn_WATER_FLT (IN-PUT)	PE0–PE3	Water-interlock fault input (one of the 12 GateDriverStatus pins, GDS?). Polarity-configurable (CONFig:FaultPolarity:WATER), routes to SM_FAULT_GENERAL (system-wide full stop) with TMP_FLT below.
XRn_TMP_FLT (INPUT)	PE4–PE7	Temperature-interlock fault input, same 12-pin GateDriverStatus group. Polarity-configurable (CONFig:FaultPolarity:TEMP), also routes to SM_FAULT_GENERAL.

Signal (direction)	Pins (XR1-4)	Purpose / commands
XRn_ENERPRO_FLT (INPUT)	PE8-PE11	Enerpro fault input, same 12-pin group. Polarity-configurable (CONFig:FaultPolarity:ENERPRO) – unlike Water/Temp, reclassified to its own per-channel SM_FAULT_ENERPRO (individual disable + derate + ramp, survivors keep running), not a full system stop.
XRn_OCP (INPUT)	PF4, PF8, PF12, PF5	Per-channel overcurrent fault input (note the irregular pin order – matches the firmware’s own kOcpPin[]). Polled, not interrupt-driven – these four pins share EXTI line numbers already claimed by the GateDriverStatus group above, a real hardware conflict, not a preference. Polarity-configurable (CONFig:FaultPolarity:OCP); routes to per-channel SM_FAULT_OVERCURRENT, same individual-disable-and-derate response as Enerpro.

11.3 System-wide signals (not per-channel)

Signal (direction)	Pin	Purpose / commands
External enable (INPUT)	PF13	Operator/facility permissive interlock. EXTERNAL:ENABLE gates ARM/a fire attempt on this reading HIGH – defaults ON at every boot as of Rev D (Section 10’s own revision-table entry). Raw level: EXTERNAL:INPUT?.

Signal (direction)		Pin	Purpose / commands
External (INPUT)	trigger	PF15	“Fiber.Enable” in docs/pin_mapping_v4.csv. A genuine LOW-to-HIGH transition fires a shot while ARMED – EXTERNAL:TRIGGER, also defaults ON as of Rev D. Edge-triggered, not level-triggered; see Step 8b, Section 9. Raw level: EXTERNAL:TRIGGER:INPUT?.
Native fault (INPUT)	HRTIM	PC10	HRTIM1_FLT6 – a real hardware fault input wired directly into the HRTIM peripheral, active-low. Forces every fault-enabled channel’s output to a safe level <i>autonomously, in silicon</i> , no CPU/interrupt/polling latency, and keeps working even if the firmware is completely hung – the only signal in this whole reference that doesn’t depend on the firmware running correctly to protect the hardware.

11.4 Diagnostic loopback outputs (bench-only, not Transrex-facing)

Four generic, software-driven outputs used to exercise the external enable/trigger inputs above from the serial console alone, without a hand-operated bench jumper – referenced by Section 12’s own entries and by python/run_simulator_validation.py’s automated campaign. Loop the relevant pin directly to PF13 or PF15 with a bench wire to use these.

Command (direction)	Pin	Typical use
DIAGnostic:GP0ut09 (OUTPUT)	PG8	Generic diagnostic output, no fixed loopback target – available for ad-hoc bench use.
DIAGnostic:GP0ut10 (OUTPUT)	PG9	Same as GP0ut09.
DIAGnostic:GP0ut11 (OUTPUT)	PD0	Loop to PF13 to drive the external-enable interlock entirely from software – precise, repeatable timing instead of a hand-operated switch.
DIAGnostic:GP0ut12 (OUTPUT)	PD1	Loop to PF15 to drive the external-trigger input the same way.

12 Troubleshooting

Symptom	Cause / action
stm32flash reports Failed to init device, timeout. before any write occurs	Not a firmware issue – this probe doesn't depend on what's running on the chip. Check board power, serial wiring (TX/RX not crossed, common GND), and that no other program has the port open.
BOOT gets no reply, but power/wiring check out	The firmware currently on the board may predate the BOOT command (run the bootstrap flash, Section 5) or may be a board affected by the Go-jump issue in Section 6.1 – power-cycle once, then retry.
Verification (Section 7) fails after a serial reflash	Power-cycle the board once and repeat verification. If it then succeeds, this is the regression described in Section 6.1 – report it. If it still fails, treat as a failed flash and repeat Section 6 from the start.
ARM returns ERR 13/ERR 15 (Section 9)	Reworked Rev D: the reply itself now names the cause – ERR 13 Can't ARM -- not currently IDLE (clear a fault or DISARM first); ERR 15 External enable interlock not satisfied -- PF13 reads LOW (see Step 7's own note, and EXTERNAL:INPUT?); or ERR 13 Channel N's ENA_OUT/ CONTACT_OUT not both set (that channel's XREX:CHANNEL: ENAOut/CONTACTOut aren't both asserted – SOURCE:ENABLE alone is not sufficient). Only the first failing precondition is reported per attempt; fix it and re-send ARM to see the next one if any.
ARM returns ERR 15 unexpectedly, on hardware with nothing physically wired to PF13	New Rev D: EXTERNAL:ENABLE now defaults ON at every boot. Whether this blocks ARM on unwired hardware depends on how that specific board's PF13 rests with nothing connected – check EXTERNAL:INPUT? directly rather than assuming. If it reads LOW and no real interlock is available for this session, send EXTERNAL:ENABLE 0 first (see Step 7's own note).

Symptom	Cause / action
SHOT:START returns ERR 12 (Section 9)	SHOT:TIMing was never successfully sent this boot, or was rejected – as of Rev C it takes three arguments (SHOT: TIMing itself will report the same ERR 12 with “needs three arguments” if only two were sent, the pre-Rev-C form).
A channel that should be silent shows real output or logged data (Section 9)	Confirm it was explicitly disabled THIS session (Step 4) – SHOT:START drives every <i>currently-enabled</i> channel, not just the ones just configured, and enable state is never implicitly reset between shots or between operators.
The external trigger (Step 8b) doesn’t fire anything, with no error reported at all	New Rev D. A triggered fire is a <i>silent</i> no-op on failure – check, in order: (1) STATE? is ARMED, not some other state (only fires from ARMED – see Section 10’s translation table); (2) SHOT:TIMing was actually sent this boot (the same RAM-only config SHOT:START needs – a fresh reflash wipes it, a real cause found live this same day); (3) EXTERNAL:TRIGger:INPut? is genuinely transitioning LOW then HIGH, not just sitting HIGH (edge-triggered, per Step 8b’s own note); (4) the external-enable interlock, if on, is satisfied (EXTERNAL:INPut?). SYS:EVLG? (Section 10) settles which of these it was after the fact – a genuine attempt that failed the interlock check still leaves no trace either way, exactly like the command-driven path’s own re-check silently staying ARMED, so ruling out (1)-(3) first is the faster path.

13 References

- AGENTS.md – project orientation and current functionality.
- docs/command_reference.md – full serial command protocol, including TABLE:*/FIRE (Section 8) and SOURCE:*/SHOT:*/LOG:*/CHANNEL:*/XREX:* (Section 9).
- docs/serial_reflash_guide.md – detailed reflash mechanism, including the Go-jump bug referenced in Section 6.1.

- docs/test_protocol.md – bench-level verification, including waveform checks beyond this SOP’s acceptance criteria.
- python/pfm_table_upload.py – reference table builder used in Section 8; see its own header comment for what each PROFILE value builds.
- python/wham_console.py – the maintained interactive operator console; its shot/refire commands apply every step of Section 9 automatically, with guided prompts and automatic plotting – the recommended tool for routine bench work, with Section 9 as the reference for what it does under the hood.
- docs/telemetry.md – the !EVT stream and flight recorder’s own design writeup (wire format, buffering, the packet-framing invariant); Section 10 above is this SOP’s operator-facing companion to it.
- docs/changelog.txt – dated development history.